DAT200 – Applied Machine Learning

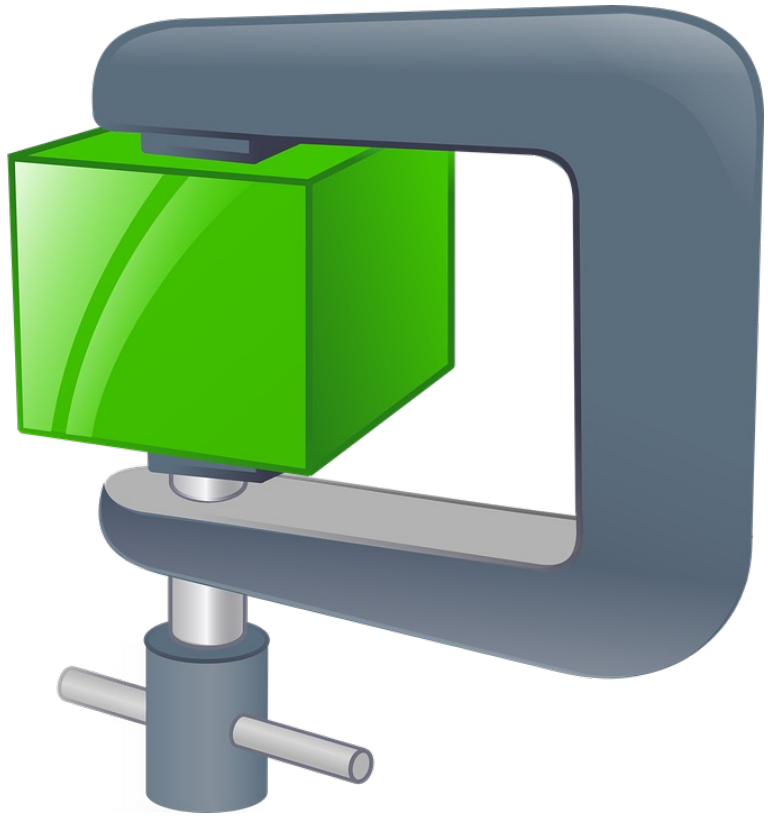
Chapter 5 in “Python Machine Learning” book

Compressing Data via Dimensionality Reduction

Feature extraction / feature selection

- Earlier, we learned about techniques for **feature selection** (e.g. with SBS/SFS) where we selected a **subset of the original features** to reduce the dimensionality of the data
- Here, we learn how to reduce the dimensionality of a dataset by **feature extraction;** techniques for **computing new features** from the existing features
- Feature extraction algorithms **transform** and/or **project** the dataset onto a **new feature subspace**
- Feature extraction can be thought of as a way of **data compression** with the goal of maintaining and/or enhancing the **most relevant information** in a dataset
- In practice, feature extraction
  - is not only attractive to reduce the required **storage space** or increase the **computational efficiency** of the learning algorithm
  - can also improve the **predictive performance** by avoiding the curse of dimensionality, especially when working with **non-regularized** models

Compression



Resources

- Python Machine Learning 3rd, Chapter 5, pages 145 – 168,

- Jupyter notebooks
  - Chapter 5 - PCA and Chapter 5 - LDA in GitLab
  - Code with the book: https://github.com/rasbt/python-machine-learning-book-3rd-edition/blob/master/ch05/ch05.ipynb

- PCA in scikit-learn: http://scikit-learn.org/stable/modules/decomposition.html#principal-component-analysis-pca

- LDA/QDA in scikit-learn: http://scikit-learn.org/stable/modules/lda_qda.html

Topics of Ch. 05 – Compressing Data via Dimensionality Reduction

- **Principal Component Analysis** (**PCA**) for unsupervised data compression
- **Linear Discriminant Analysis** (**LDA**) as a supervised dimensionality reduction technique for maximizing class separability

Unsupervised learning

- The goal of **unsupervised learning** is to **identify hidden patterns** or **structure** in the data
- We only work with the design matrix $\boldsymbol{X}$
- We do not know or do not use the labels/outcome $\boldsymbol{y}$

UNSUPERVISED DIMENSIONALITY REDUCTION USING PRINCIPAL COMPONENT ANALYSIS

Principal Component Analysis (PCA)

- **Explorative multivariate** data analysis method
  - Versatile method for almost all types of data to obtain an **overview** (e.g. in an early phase of investigation)
- Particularly suitable in situations with **lots of data**, and **little prior information** available
- We can **reduce the dimensionality of our data** through feature extraction and **data compression** by finding new features that contain **the most relevant information**
- PCA extracts the **main information** (**systematic variation**) – the dominant **patterns/directions** - in the data

Pearson, K., *Philosophical Magazine,* 1901, ***2*** (11): 559–572.

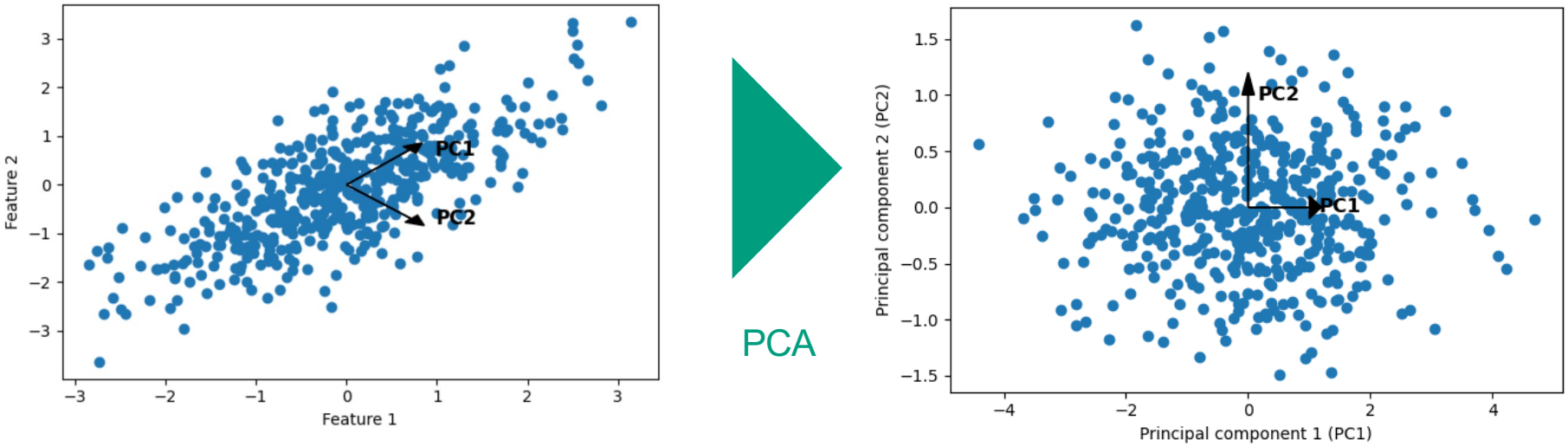
Principal Component Analysis (PCA)

- PCA in a nutshell:
  - Finds the **directions/axes** of **maximum variance** (principal components) in high-dimensional data and and **projects** it onto a **new subspace** with (preferably) fewer dimensions than the original one
  - The principal components are **orthogonal axes** that form this new subspace

- **Visualisation:** Present results graphically for interpretation of the **variance** in the data
  - Information on **samples** and **variables**
  - Knowledge on how **samples** are distributed (interpretation of patterns using background information)
  - Knowledge on how **variables** contribute to the variance in the data
  - Generate hypotheses and ideas for further investigation

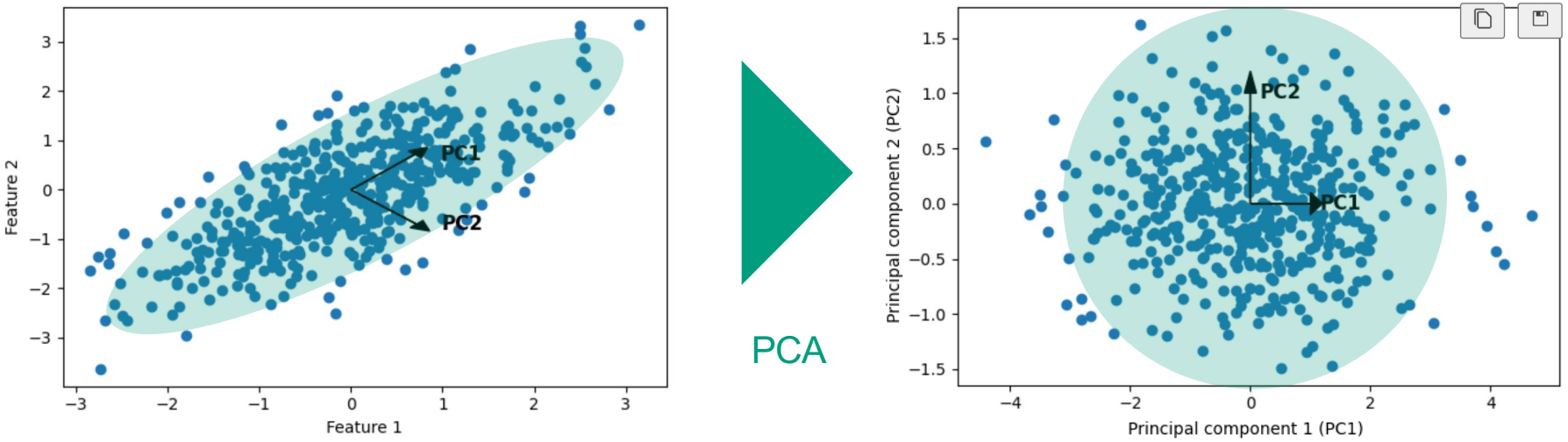
Principal Component Analysis (PCA)

- An unsupervised **linear transformation** technique often used for **dimensionality reduction**
- **Identifies patterns/structure** in the data based on the **covariance** (centered data) or **correlation** (standardized data)



Principal Component Analysis (PCA)

- An unsupervised **linear transformation** technique often used for **dimensionality reduction**
- **Identifies patterns/structure** in the data based on the **covariance** (centered data) or **correlation** (standardized data)



Principal Component Analysis (PCA)

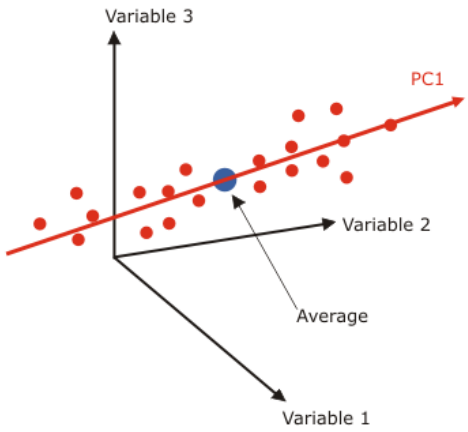
- Applied as a **pre-processing technique** for other models
  - Make high-dimensional data suitable for visualization
  - May help to prevent overfitting (especially for models without a regularization option)
  - Denoising of the original data – PCA as a filter to get rid of noise
  - Data compression/dimensionality reduction (as often used in ML) – use the principal components instead of the original features as inputs to *classifiers*, *regression* or *clustering*

- Widely used across different fields, for e.g.:
  - Explorative data analysis (EDA)
  - Noise reduction in signal data (e.g. stock market prices)
  - Analysis of genome and gene expression data (very high-dimensional data)

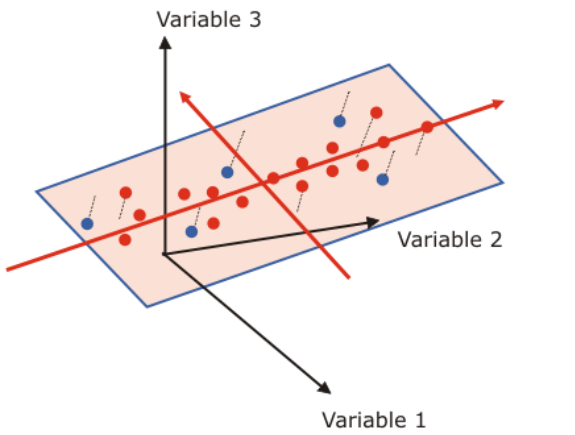
PCA basics – description of the method

- In the figures below, each row/sample is represented as a point in the three-dimensional coordinate system
- **Note**: For illustration only, this is not a typical situation for the use of PCA, since we have only 3 variables in the data set, and PCA is used for dimensionality reduction
- For matrices with more than 3 variables PCA cannot be visualised graphically, but the mathematics behind is the same

The First Principal Component



PCs 1 and 2: a new window for looking into multidimensional space



Figures from CAMO Unscrambler software help documentation

PCA basics – description of the method

- **Step 1**: Center the data (subtract the averages from their corresponding variables). This corresponds to moving the origin of the coordinate system or vector space to the average data point
- **Step 2**: Search for the direction in space that has the **largest** variance ➔ component (PC) 1
- **Step 3**: Search for the direction in space that has **largest** variance **AND** is **orthogonal** to PC 1 ➔ PC 2
- **Step 4**: Search for the direction in space that has **largest** variance **AND** is **orthogonal** to PC 1 **and** PC 2 ➔ PC 3
- **Step 5**: etc.

PCs 1 and 2: a new window for looking into multidimensional space

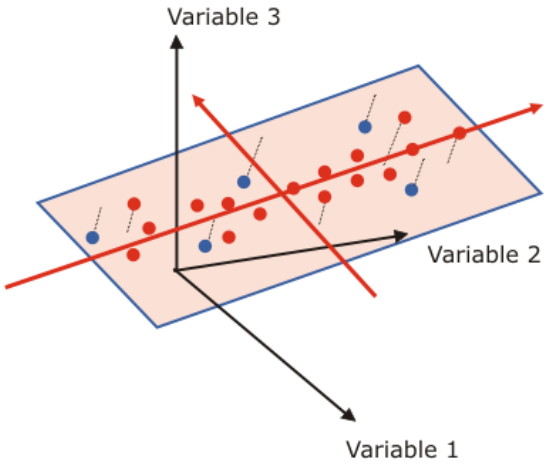


Figure from CAMO Unscrambler software help documentation

The main steps behind principal component analysis

The main steps behind PCA from eigenvalue decomposition

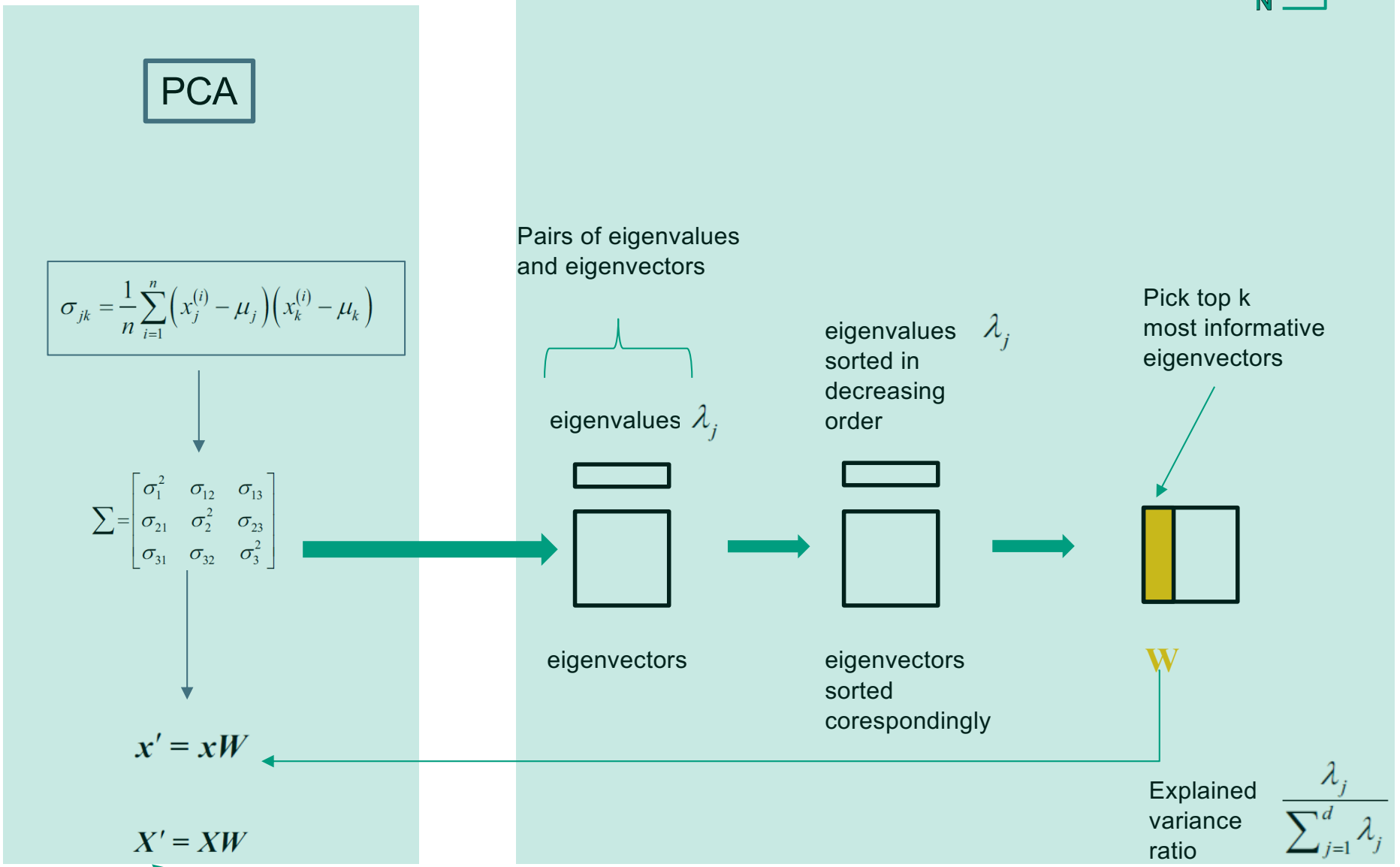
Equivalent to SVD on **X**

1) Standardize the d-dimensional dataset (if needed).
2) Construct the covariance/correlation matrix **($\boldsymbol{\Sigma} = \frac{1}{n-1}\boldsymbol{X}^T\boldsymbol{X}$)**
3) Decompose the covariance matrix into its eigenvectors and eigenvalues.
4) Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors.
5) Select k eigenvectors which correspond to the k largest eigenvalues, where k is the dimensionality of the new feature subspace ($k < d$).
6) Construct a projection matrix $\boldsymbol{W}$ from the "top" k eigenvectors.
7) Transform the d-dimensional input dataset $\boldsymbol{X}$ using the projection matrix $\boldsymbol{W}$ to obtain the new k -dimensional feature subspace.

Score vectors (coordinates of the samples in the new feature space): $\boldsymbol{T} = \boldsymbol{X}\boldsymbol{W}$

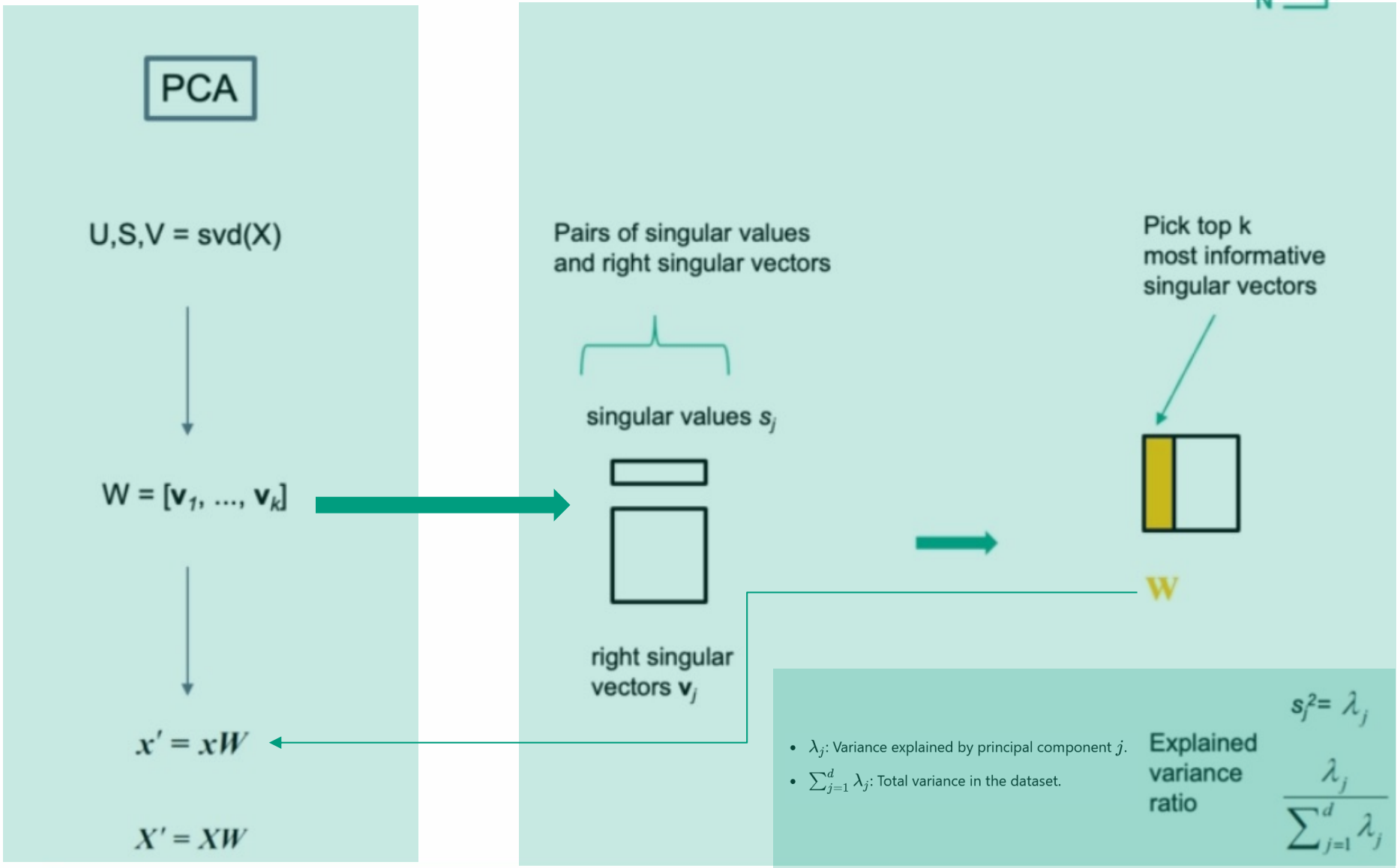
Centering but not standardization is included in `PCA` from `sklearn`

Principal component analysis by eigenvalue decomposition



Score vectors (coordinates of the samples in the new feature space), commonly denoted ***T***

Principal component analysis by singular value decomposition



Principal Component Analysis (PCA)

- Decomposition of a large dataset into a subspace of **latent variables** (scores, *T* and loadings, *P*) representing the main features of **variance**

$$X = \bar{x} + \sum_{a=1}^{A} t_{Xa} p_a{}' + E_A =$$
$$\bar{x} + T_{XA} P_A{}' + E_A$$

- *A* denotes the number of Principal Components (PCs) used, E_A is the error term using *A* PCs
- The score vectors are orthogonal

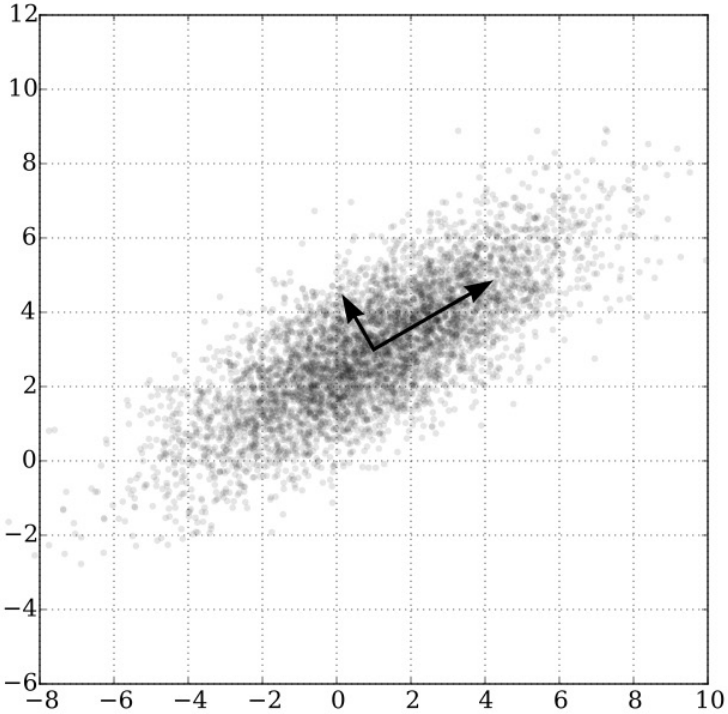
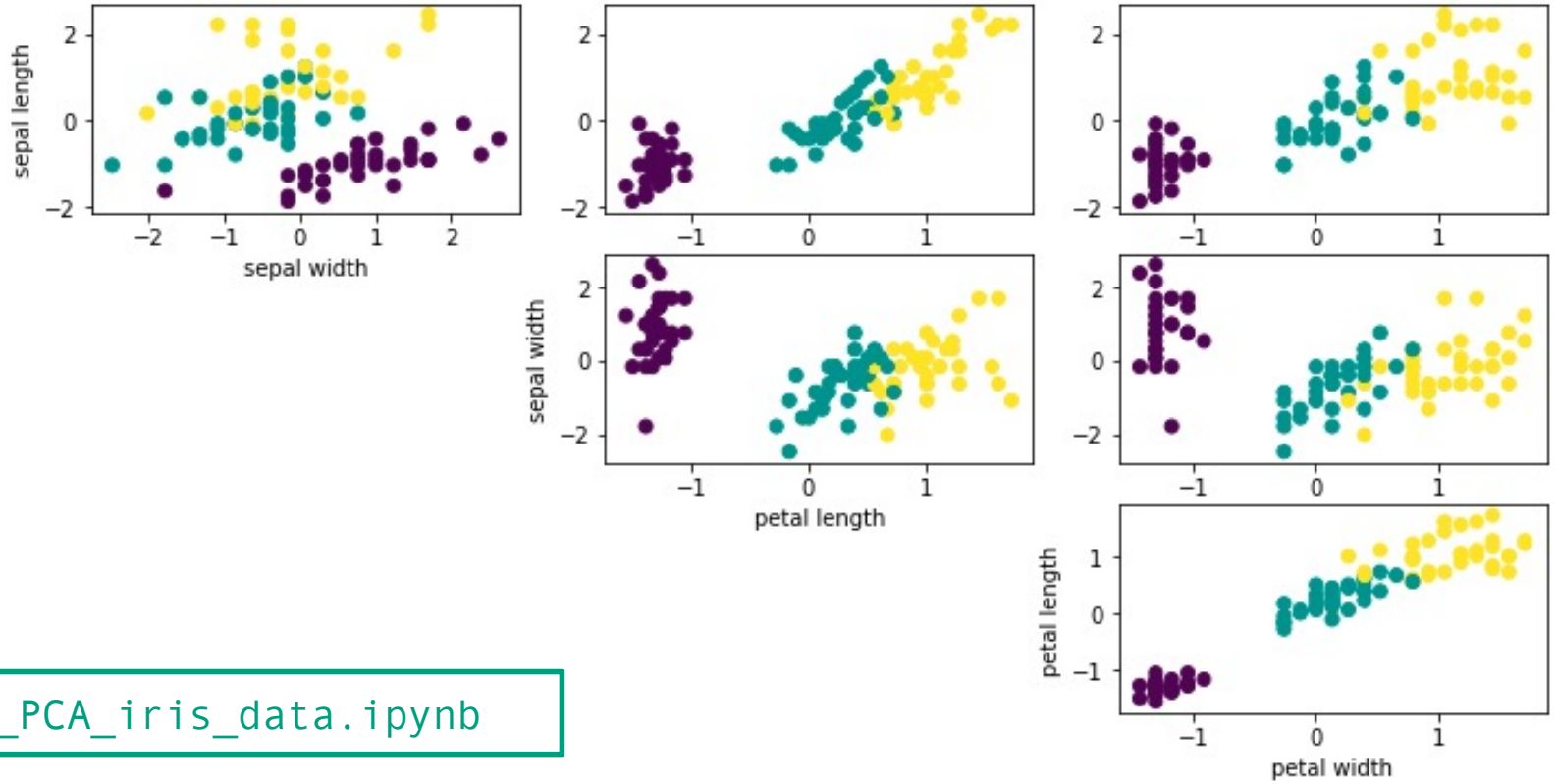


Figure by Nicoguaro - Own work, CC BY 4.0, https://commons.wikimedia.org/w/index.php?curid=46871195

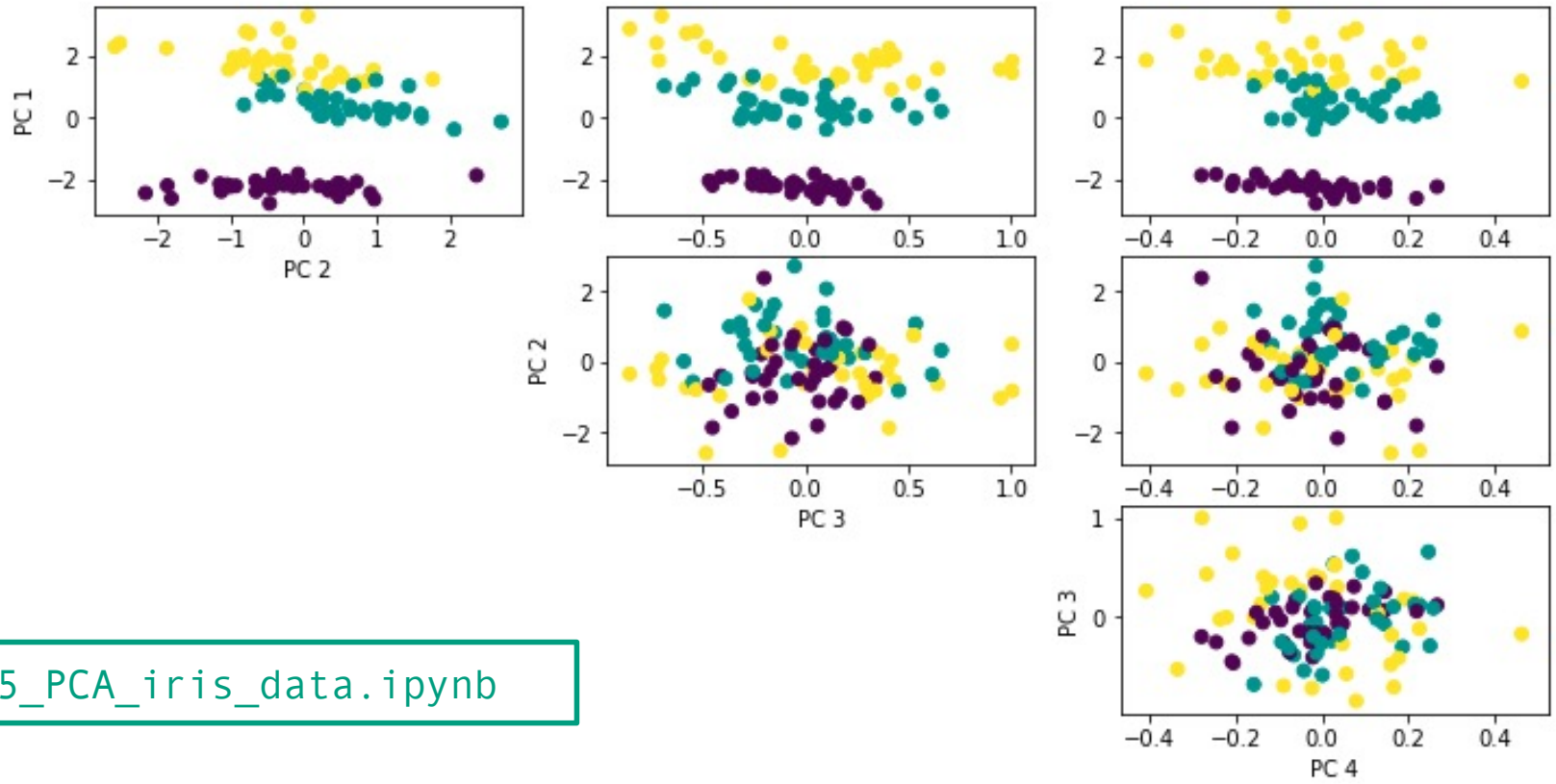
Pearson, K., *Philosophical Magazine,* 1901, ***2*** (11): 559–572.

Consider all feature pairs for the Iris data —which are best?



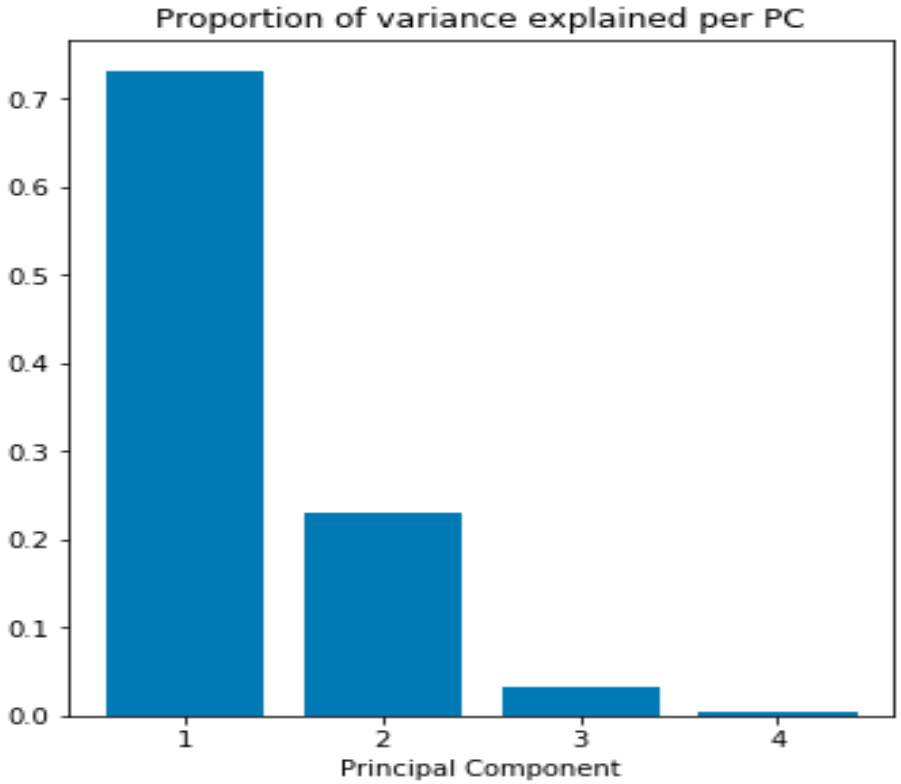
`05_PCA_iris_data.ipynb`

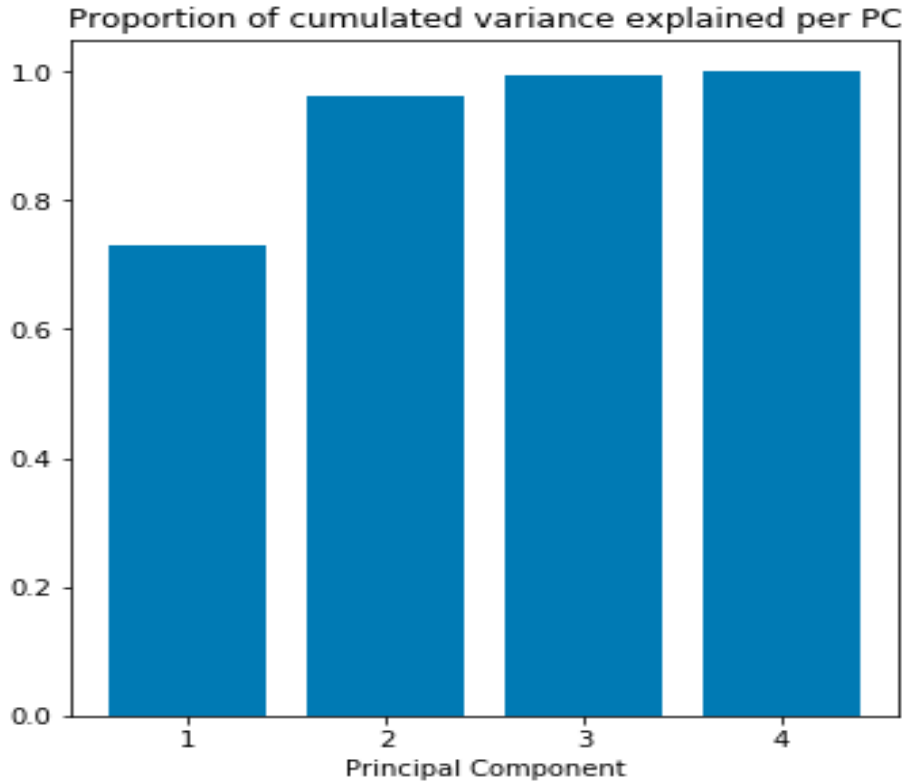
PCA automatically finds the optimal coordinate combinations



`05_PCA_iris_data.ipynb`

PCs ordered by the explained variance



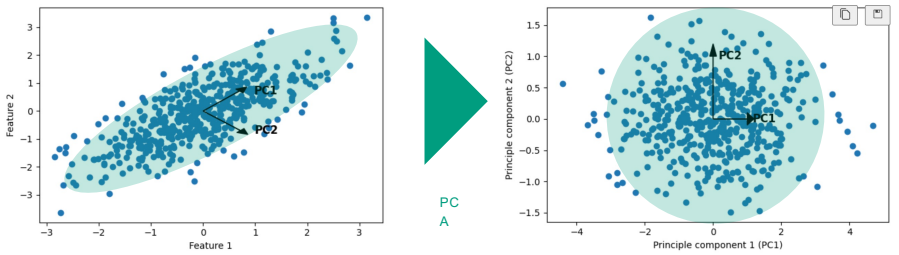


`05_PCA_iris_data.ipynb`

Principal Component Analysis (PCA)

The algorithm is explained in:

`05_PCA.ipynb`



Example applications:

`05_PCA_wine_explained_variance.ipynb` → Feature extraction / importance

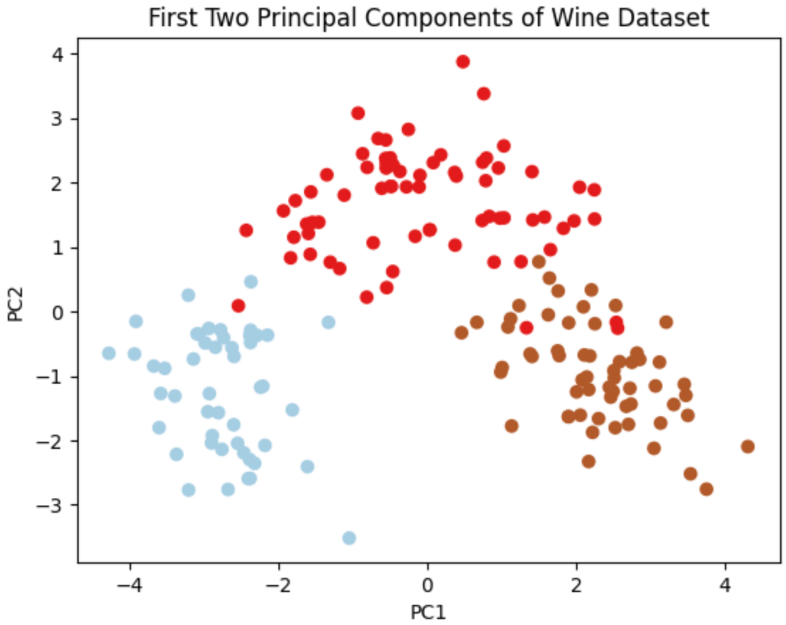
`05_PCA_handwritten_digits.ipynb` → Visualization

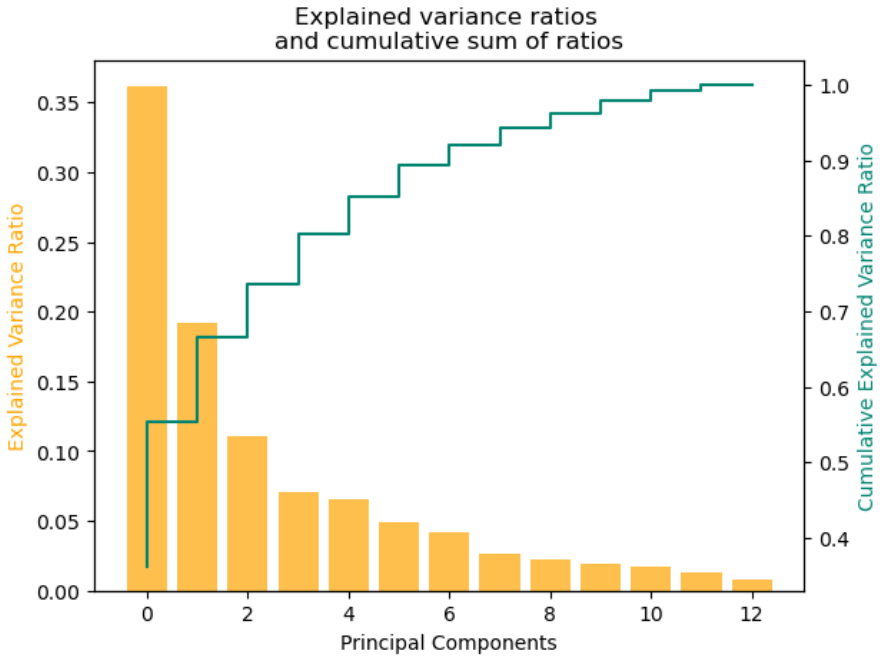
`05_PCA_logreg_wine.ipynb` → Preprocessing / feature extraction

Principal Component Analysis (PCA)

`05_PCA_wine_explained_variance.ipynb`

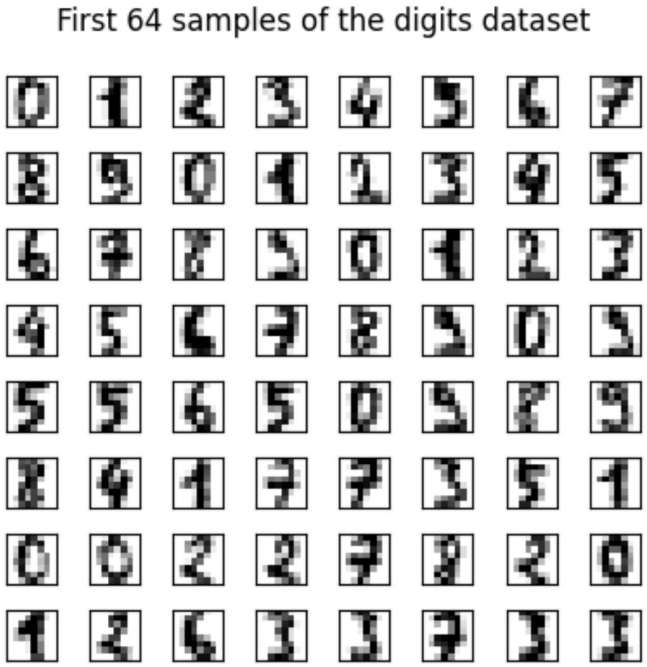
Explained variance ratio = $\frac{\lambda_j}{\sum_{j=1}^{m} \lambda_j}$



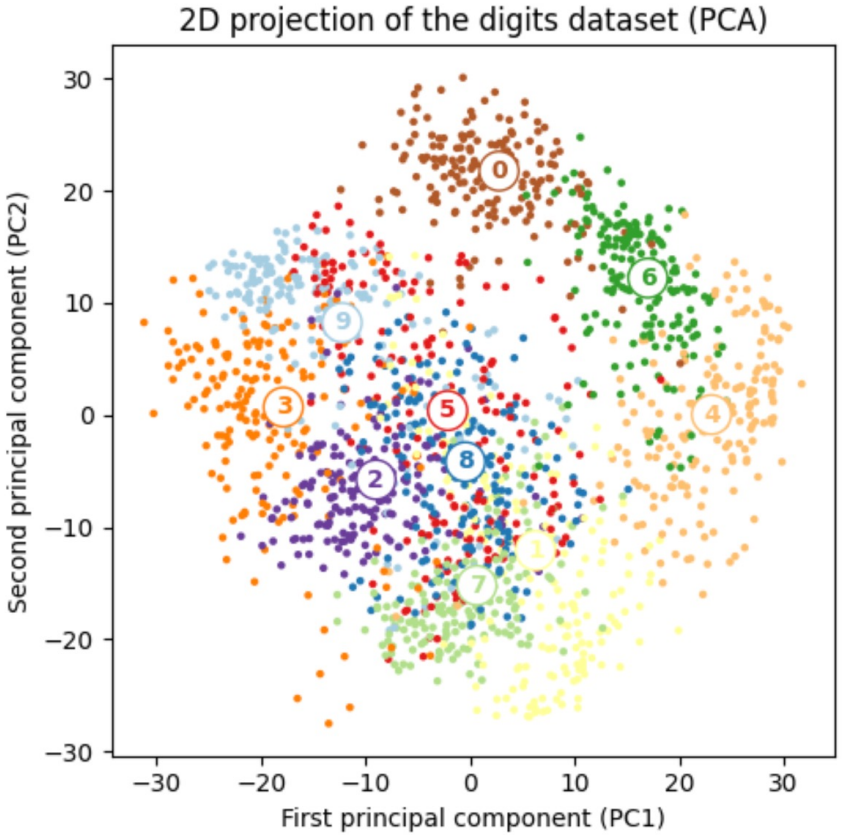


Principal Component Analysis (PCA)

`05_PCA_handwritten_digits.ipynb`

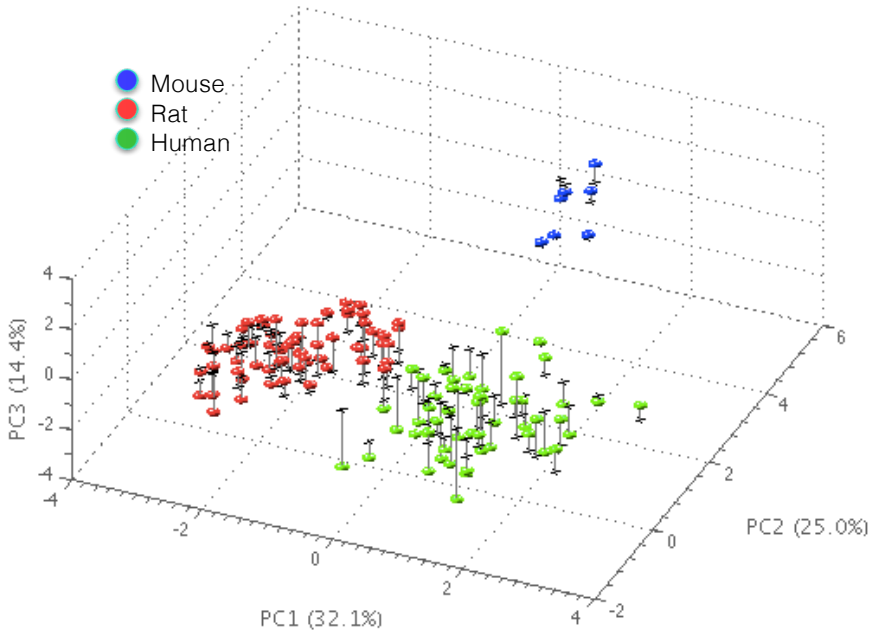


PCA
(k=2)



Example: Parameters of a dynamic model of a heart cell

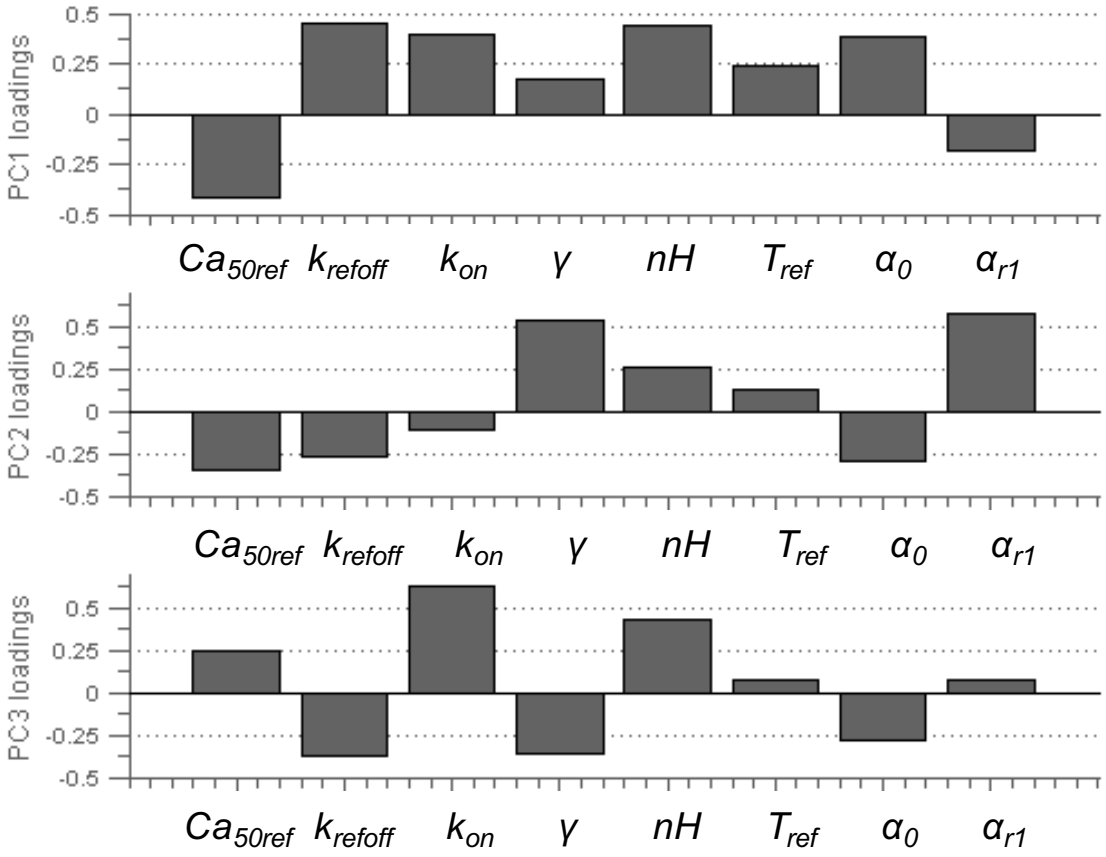
Scores from PCA on fitted parameter values for a dynamic model of heart cell contraction:



The separation formed a basis for quantification of inter-species differences in heart cell contraction

Tøndel, K. *et al*, *J. Physiol.*, 2015, 593(5), 1083-111.

PCA loadings



The PCA loadings (***P***) tell us which parameters (original features related to biophysical mechanisms) contribute the most to the latent variables/separation between the species

$$X = \bar{x} + \sum_{a=1}^{A} t_{Xa} p_a' + E_A =$$

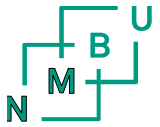
$$\bar{x} + T_{XA} P_A' + E_A$$

Tøndel, K. *et al*, *J. Physiol.*, 2015, 593(5), 1083-111.

Why use PCA? Advantages of PCA

- **Reduces** the **number of features** which can lead to
  - a significant speed-up of the model training and prediction
  - a reduction in the required memory to store the data (data compression)
  - extraction of the most important information in the data
- May **reduce noise** in the data (e.g. common for signal data) which may improve the performance of an ML model that is sensitive to noise
- Makes it possible to **visualize high-dimensional data sets** using only the features with the highest variance

Why not always use PCA? Disadvantages of PCA



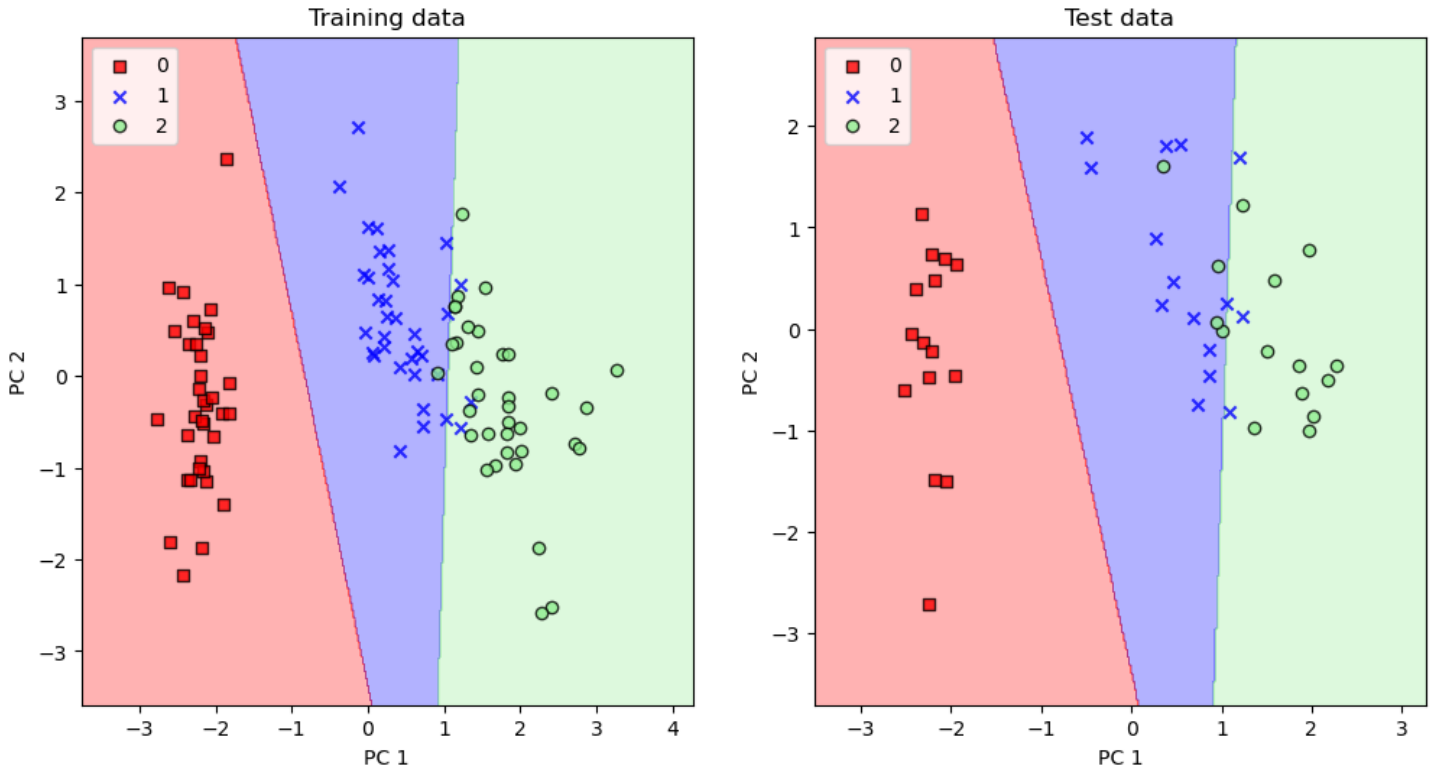
- **Principal components** are difficult to interpret (no obvious connection to the original features)
- **Principal components** with a **low explained variance** may, in fact, be important for e.g. class separation. PCA detects features with high variance. It doesn't use class labels (is unsupervised). In case highly discriminative features (separating classes) have low variance, PCA discards important information
- **PCA** is a linear model, but relations between features may be more complex than linear combinations of the original features
- The **PCA** result highly **depends** on the **scaling of the variables**. For example, going from *km* to *cm* for a length feature will increase the importance of the feature a lot (if we don´t standardise our data)

Dimensionality reduction before supervised learning

- Perform PCA on the original data
- Select the necessary number of components to explain enough variance
- Discard all other components
- Perform supervised learning for classification/regression only on the retained components
- This avoids confusing/stressing the supervised learning with features that carry little information

PCA as a preprocessing technique

- PCA for feature extraction
- Logistic regression for classification

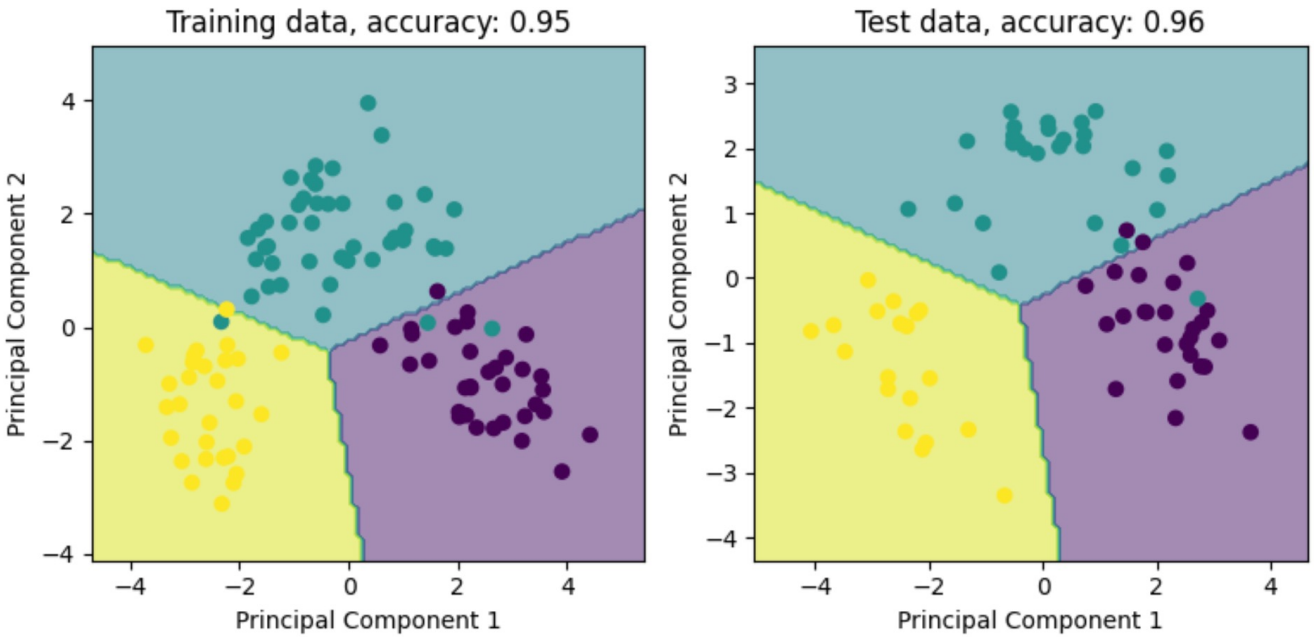


`05_PCA_iris_data.ipynb`

PCA as a preprocessing technique

`05_PCA_logreg_wine.ipynb`

- PCA for feature extraction
- Logistic regression for classification



Principal Component Analysis in other fields

All based on the same principle and mathematics!

- **Eigenvalue decomposition** (for symmetric matrices) of $\boldsymbol{X^T X}$ ($\boldsymbol{X}$ is centered)
- **Singular value decomposition** of $\boldsymbol{X}$ (singular values are the square roots of the eigenvalues)
- **Proper Orthogonal Decomposition** (POD): Surrogate models and reduced order models (ROM) in engineering and robotics
- **Karhunen–Loève transformation**: Signal processing
- **Spectral decomposition / modal analysis**: Audio and vibration signal processing, structural dynamics (e.g. eigenmodes of a vibrating drum or string)
- **Principal axis transformation:** Mechanical engineering, structural mechanics

Pearson, K., *Philosophical Magazine,* 1901, ***2*** (11): 559–572.

SUPERVISED DATA COMPRESSION USING LINEAR DISCRIMINANT ANALYSIS

Feature extraction using Linear Discriminant Analysis

- What do you think *linear discriminant analysis* aims to accomplish?
- As the name suggests, we aim to **transform** the data into a **subspace** where it is as easy as possible to "**linearly discriminate**" between samples of different classes

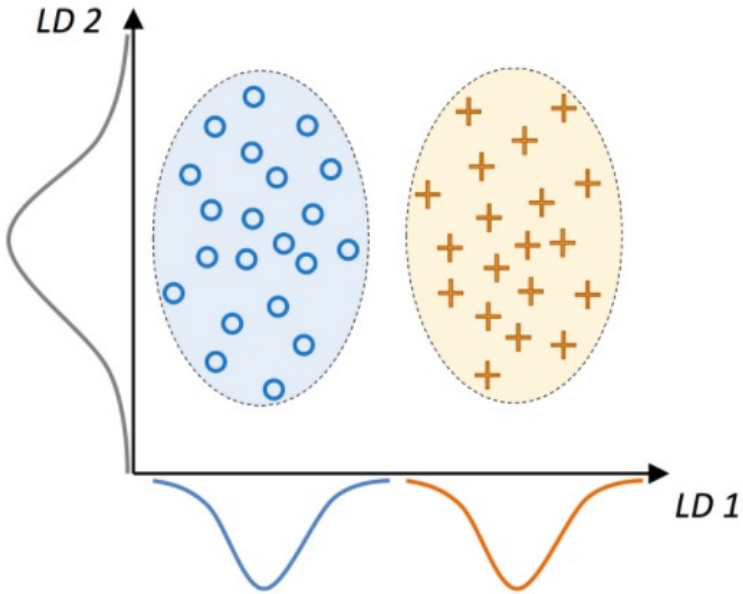
Supervised data compression via linear discriminant analysis

- **Linear Discriminant Analysis** (**LDA**) can be used as a technique for feature extraction
  - **Increases** the computational efficiency
  - **Reduces** the degree of **overfitting** due to the curse of dimensionality in non-regularized models

- The general concept behind LDA is **very similar** to PCA
  - Both LDA and PCA are **linear transformation techniques** that can be used to reduce the number of dimensions in a dataset by **feature extraction**
    - PCA is an **unsupervised** algorithm
    - LDA is a **supervised** algorithm
  - PCA attempts to find the **orthogonal component axes** of **maximum variance** in a dataset
  - LDA finds the **feature subspace** that optimizes linear **class separability**
  - Both decompose matrices into **eigenvalues** and **eigenvectors**, which will form a new lower-dimensional feature space.

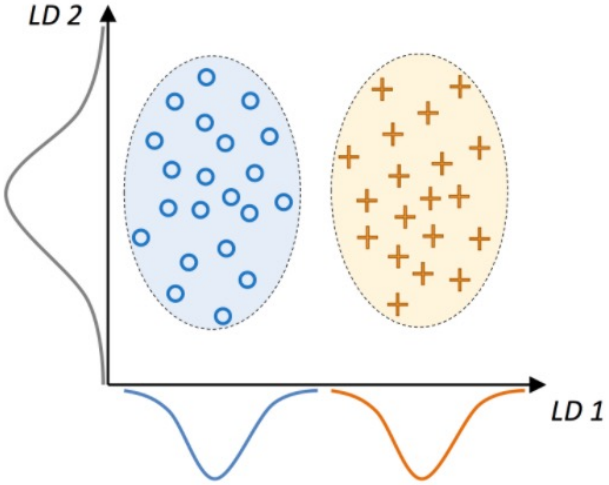
Feature extraction using Linear Discriminant Analysis

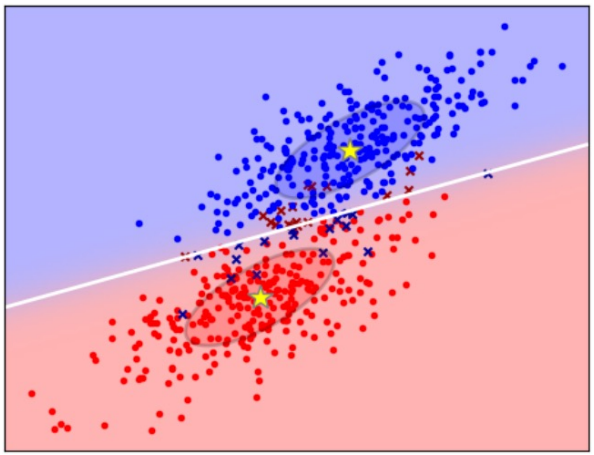
- The concept of LDA
- In the binary classification example to the right a dataset has been projected onto two *linear discriminants*.
- Which of the **LD**'s would you choose to classify the dataset?



Principal component analysis versus linear discriminant analysis

Concept of LDA





- Linear discriminant LD 1, would separate the two normally distributed classes well
- LD 2 captures a lot of the variance in the dataset but
  - **does not** capture any of the class-discriminatory information
  - **fails** as a good linear discriminant
- LDA assumes that
  - the data is **normally distributed**
  - classes have **identical covariance matrices**
  - features are **statistically independent** of each other
- Even if one or more of those assumptions are (slightly) **violated**, LDA can still work reasonably well for dimensionality reduction

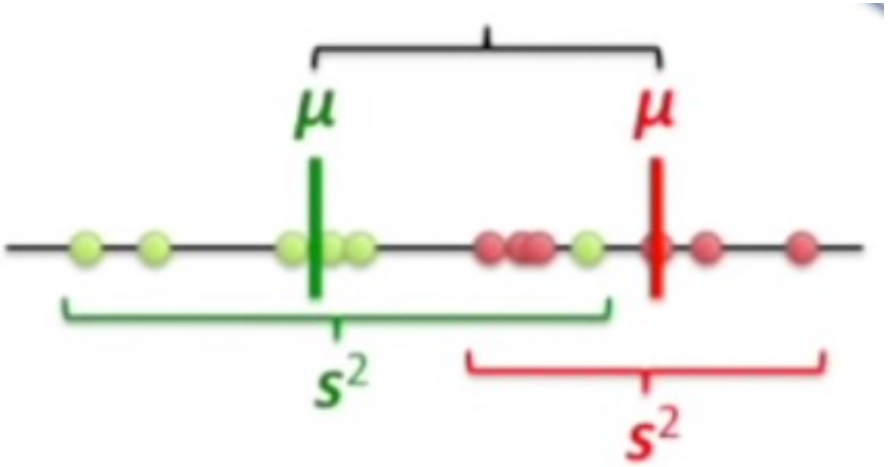
https://scikit-learn.org/stable/auto_examples/classification/plot_lda_qda.html

Inner workings of LDA

Introduction - LDA

To project our data into a new axis we want to:

Maximize the distance between the means



Minimize the variation (scatter) between the two categories

Inner workings of LDA

- **Brief summary of the main steps that are required to perform LDA for a c-groups problem:**
  - **NOTE**: Standardization of the d-dimensional dataset (d is the number of original features) is not critical.

1. For each class, compute the associated d-dimensional mean vector.
2. Construct the between-groups scatter matrix $\boldsymbol{S}_B$ and the within-groups scatter matrix $\boldsymbol{S}_W$
3. Compute the eigenvectors and corresponding eigenvalues of the matrix $\boldsymbol{S}_W^{-1}\boldsymbol{S}_B$
4. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors.
5. Choose the $k \leq c$ - 1 eigenvectors that correspond to the k largest eigenvalues to construct a $d \times k$-dimensional transformation matrix $\boldsymbol{W}$; the k most dominant eigenvectors are the columns of this matrix.
6. Transform the samples into the new features by multiplication with the transformation matrix $\boldsymbol{W}$

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

`LDA_from_scratch.ipynb`

Performing LDA from scratch

Principal component analysis versus linear discriminant analysis

Principal component analysis versus linear discriminant analysis

1. Compute Class Mean

- c → Total number of classes
- m_i → Mean vector of class i
- n_i → Number of samples in class i
- D_i → Set of samples in class i
- x → Feature vectors

2. Compute Within-Class Scatter

- S_W → Within-class scatter matrix
- S_i → Scatter matrix of class i
- x → Feature vector
- m_i → Mean vector of class i
- c → Total number of classes

3. Compute Between-Class Scatter

- S_B → Between-class scatter matrix
- n_i → Number of samples in class i
- m_i → Mean vector of class i
- m → Overall mean vector

LDA

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m$$

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x}\in D_i}^{c}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^T$$

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i$$

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i(\boldsymbol{m}_i-\boldsymbol{m})(\boldsymbol{m}_i-\boldsymbol{m})^T$$

4. Find Best Separation Directions

$$\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$$

Select Top *k* Discriminants to Form ***W***

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$: to find the transformation matrix ***W***

Max number of LD's: *c* - 1

5. Transform data

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Principal component analysis versus linear discriminant analysis

LDA

1. Compute Class Mean

- c → Total number of classes
- m_i → Mean vector of class i
- n_i → Number of samples in class i
- D_i → Set of samples in class i
- x → Feature vectors

$$\boldsymbol{m}_i = \frac{1}{n_i} \sum_{\boldsymbol{x} \in D_i}^{c} \boldsymbol{x}_m$$

2. Compute Within-Class Scatter

- S_W → Within-class scatter matrix
- S_i → Scatter matrix of class i
- x → Feature vector
- m_i → Mean vector of class i
- c → Total number of classes

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x} \in D_i}^{c} (\boldsymbol{x} - \boldsymbol{m}_i)(\boldsymbol{x} - \boldsymbol{m}_i)^T$$

Measuring the within-class variance

Measures how spread out the data points are within each class

$$\boldsymbol{S}_W = \sum_{i=1}^{c} \boldsymbol{S}_i$$

3. Compute Between-Class Scatter

- S_B → Between-class scatter matrix
- n_i → Number of samples in class i
- m_i → Mean vector of class i
- m → Overall mean vector

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i (\boldsymbol{m}_i - \boldsymbol{m})(\boldsymbol{m}_i - \boldsymbol{m})^T$$

4. Find Best Separation Directions

$$\boldsymbol{S}_w^{-1} \boldsymbol{S}_B$$

Select Top *k* Discriminants to Form ***W***

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1} \boldsymbol{S}_B$: to find the transformation matrix ***W***

Max number of LD's: *c* - 1

6. Transform data

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Principal component analysis versus linear discriminant analysis

1. Compute Class Mean

- c → Total number of classes
- m_i → Mean vector of class i
- n_i → Number of samples in class i
- D_i → Set of samples in class i
- x → Feature vectors

2. Compute Within-Class Scatter

- S_W → Within-class scatter matrix
- S_i → Scatter matrix of class i
- x → Feature vector
- m_i → Mean vector of class i
- c → Total number of classes

3. Compute Between-Class Scatter

- S_B → Between-class scatter matrix
- n_i → Number of samples in class i
- m_i → Mean vector of class i
- m → Overall mean vector

LDA

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m$$

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x}\in D_i}^{c}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^T$$

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i$$

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i(\boldsymbol{m}_i-\boldsymbol{m})(\boldsymbol{m}_i-\boldsymbol{m})^T$$

Measures how far apart the class centers are

4. Find Best Separation Directions

$$\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$$

Select Top *k* Discriminants to Form ***W***

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$: to find the transformation matrix ***W***

Max number of LD's: *c* - 1

5. Transform data

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Principal component analysis versus linear discriminant analysis

1. Compute Class Mean

- c → Total number of classes
- m_i → Mean vector of class i
- n_i → Number of samples in class i
- D_i → Set of samples in class i
- x → Feature vectors

2. Compute Within-Class Scatter

- S_W → Within-class scatter matrix
- S_i → Scatter matrix of class i
- x → Feature vector
- m_i → Mean vector of class i
- c → Total number of classes

3. Compute Between-Class Scatter

- S_B → Between-class scatter matrix
- n_i → Number of samples in class i
- m_i → Mean vector of class i
- m → Overall mean vector

LDA

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m$$

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x}\in D_i}^{c}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^T$$

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i$$

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i(\boldsymbol{m}_i-\boldsymbol{m})(\boldsymbol{m}_i-\boldsymbol{m})^T$$

4. Find Best Separation Directions

$$\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$$

Select Top *k* Discriminants to Form ***W***

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$: to find the transformation matrix ***W***

Max number of LD's: *c* - 1

5. Transform data

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Principal component analysis versus linear discriminant analysis

1. Compute Class Mean

- m_i → Mean vector of class i
- n_i → Number of samples in class i
- D_i → Set of samples in class i
- x → Feature vectors

2. Compute Within-Class Scatter

- S_W → Within-class scatter matrix
- S_i → Scatter matrix of class i
- x → Feature vector
- m_i → Mean vector of class i
- c → Total number of classes

3. Compute Between-Class Scatter

- S_B → Between-class scatter matrix
- n_i → Number of samples in class i
- m_i → Mean vector of class i
- m → Overall mean vector

LDA

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m$$

$$\boldsymbol{S}_i = \sum_{\boldsymbol{x}\in D_i}^{c}\left(\boldsymbol{x}-\boldsymbol{m}_i\right)\left(\boldsymbol{x}-\boldsymbol{m}_i\right)^T$$

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i$$

$$\boldsymbol{S}_B = \sum_{i=1}^{c} n_i\left(\boldsymbol{m}_i-\boldsymbol{m}\right)\left(\boldsymbol{m}_i-\boldsymbol{m}\right)^T$$

4. Find Best Separation Directions

$$\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$$

Select Top *k* Discriminants to Form ***W***

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$: to find the transformation matrix ***W***

Max number of LD's: *c* - 1

5. Transform data

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Principal component analysis versus linear discriminant analysis

PCA

Mean centered data (or standard scaled) data matrix $\boldsymbol{X}$

SVD of $\boldsymbol{X}$ to find the transformation matrix $\boldsymbol{W}$

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

LDA

$$\boldsymbol{m}_i = \frac{1}{n_i}\sum_{\boldsymbol{x}\in D_i}^{c}\boldsymbol{x}_m \qquad \boldsymbol{S}_i = \sum_{\boldsymbol{x}\in D_i}^{c}(\boldsymbol{x}-\boldsymbol{m}_i)(\boldsymbol{x}-\boldsymbol{m}_i)^T$$

$$\boldsymbol{S}_W = \sum_{i=1}^{c}\boldsymbol{S}_i \qquad \boldsymbol{S}_B = \sum_{i=1}^{c} n_i(\boldsymbol{m}_i-\boldsymbol{m})(\boldsymbol{m}_i-\boldsymbol{m})^T$$

$\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$

Eigenvaluedecomp. of $\boldsymbol{S}_w^{-1}\boldsymbol{S}_B$: to find the transformation matrix $\boldsymbol{W}$

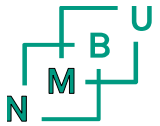
Max number of LD's: c - 1

Limits the number of features vs samples, could combine with PCA.

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

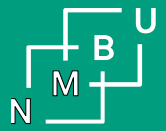
$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

LDA using scikit-learn



- LDA for feature extraction
- Logistic regression for classification
- LDA can also be used for direct prediction, since it is a classification algorithm in addition to being used for feature extraction

`LDA_and_logistic_regression_with_sklearn.ipynb`


U
B
M
N